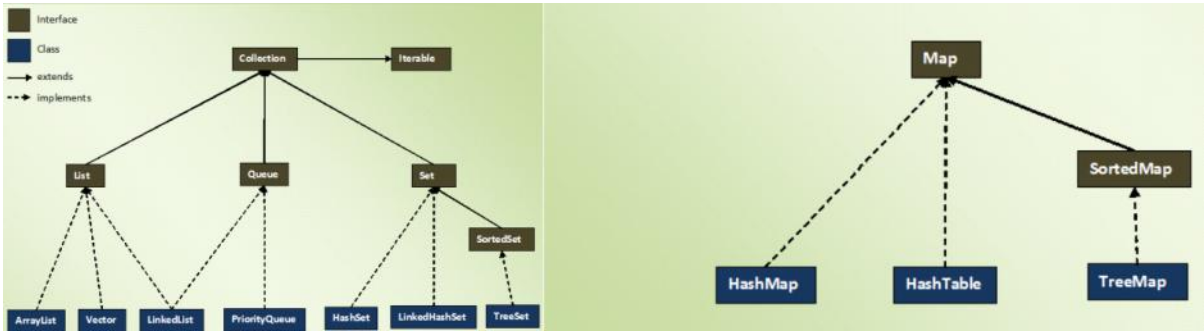
Introduction

29 July 2018 12:30



The Java Collections Framework is a class library designed to handle multiple objects, implemented in the java.util package. It stores references to other objects, representing a single unit of objects.

Iterable Interface:

- Root interface for all collection classes.
- Collection interface extends Iterable.
- Contains one abstract method: Iterator<T> iterator(), which returns an iterator over elements of type T.

Collection Interface:

- Implemented by all classes in the collection framework.
- Declares methods that every collection must have.
- Builds the foundation for the collection framework.
- Some methods: Boolean add(Object obj), Boolean addAll(Collection c), void clear(), etc.

Java Collections Cheat Sheet

Java Concept Of The Day

Basics

What is Java Collection Framework?

Java Collection Framework is a framework which provides some predefined classes and interfaces to store and manipulate the group of objects. Using Java collection framework, you can store the objects as a List or as a Set or as a Queue or as a Map and perform basic operations like adding, removing, updating, sorting, searching etc... with ease.

Why Java Collection Framework?

Earlier, arrays are used to store the group of objects. But, arrays are of fixed size. You can't change the size of an array once it is defined. It causes lots of difficulties while handling the group of objects. To overcome this drawback of arrays, Java Collection Framework is introduced from JDK 1.2.

Java Collections Hierarchy :

All the classes and interfaces related to Java collections are kept in java.util package. List, Set, Queue and Map are four top level interfaces of Java collection framework. All these interfaces (except Map) inherit from java.util.Collection interface which is the root interface in the Java collection framework.

List

Intro :

- List is a sequential collection of objects.
- Elements are positioned using zero-based index.
- Elements can be inserted or removed or retrieved from any arbitrary position using an integer index.

Popular Implementations :

- ArrayList, Vector And LinkedList

Internal Structure :

- **ArrayList :** Internally uses re-sizable array which grows or shrinks as we add or delete elements.
- **Vector :** Same as ArrayList but it is synchronized.
- **LinkedList :** Elements are stored as Nodes where each node consists of three parts – Reference To Previous Element, Value Of The Element and Reference To Next Element.

Null Elements :

- **ArrayList :** Yes
- **Vector :** Yes
- **LinkedList :** Yes

Duplicate Elements :

- **ArrayList :** Yes
- **Vector :** Yes
- **LinkedList :** Yes

Order Of Elements :

- **ArrayList :** Insertion Order
- **Vector :** Insertion Order
- **LinkedList :** Insertion Order

Synchronization :

- **ArrayList :** Not synchronized
- **Vector :** Synchronized
- **LinkedList :** Not synchronized

Performance :

- **ArrayList :** Insertion -> O(1) (if insertion causes restructuring of internal array, it will be O(n)), Removal -> O(1) (if removal causes restructuring of internal array, it will be O(n)), Retrieval -> O(1)
- **Vector :** Similar to ArrayList but little slower because of synchronization.
- **LinkedList :** Insertion -> O(1), Removal -> O(1), Retrieval -> O(n)

When to use?

- **ArrayList :** Use it when more search operations are needed then insertion and removal.
- **Vector :** Use it when you need synchronized list.
- **LinkedList :** Use it when insertion and removal are needed frequently.

Queue

Intro :

- Queue is a data structure where elements are added from one end called tail of the queue and elements are removed from another end called head of the queue.
- Queue is typically FIFO (First-In-First-Out) type of data structure.

Popular Implementations :

- PriorityQueue, ArrayDeque and LinkedList (implements List also)

Internal Structure :

- **PriorityQueue :** It internally uses re-sizable array to store the elements and a Comparator to place the elements in some specific order.
- **ArrayDeque :** It internally uses re-sizable array to store the elements.

Null Elements :

- **PriorityQueue :** Not allowed
- **ArrayDeque :** Not allowed

Duplicate Elements :

- **PriorityQueue :** Yes
- **ArrayDeque :** Yes

Order Of Elements :

- **PriorityQueue :** Elements are placed according to supplied Comparator or in natural order if no Comparator is supplied.
- **ArrayDeque :** Supports both LIFO and FIFO

Synchronization :

- **PriorityQueue :** Not synchronized
- **ArrayDeque :** Not synchronized

Performance :

- **PriorityQueue :** Insertion -> O(log(n)), Removal -> O(log(n)), Retrieval -> O(1)
- **ArrayDeque :** Insertion -> O(1) , Removal -> O(n), Retrieval -> O(1)

When to use?

- **PriorityQueue :** Use it when you want a queue of elements placed in some specific order.
- **ArrayDeque :** You can use it as a queue OR as a stack.

Set

Intro :

- Set is a linear collection of objects with no duplicates.
- Set interface does not have its own methods. All its methods are inherited from Collection interface. It just applies restriction on methods so that duplicate elements are always avoided.

Popular Implementations :

- HashSet, LinkedHashSet and TreeSet

Internal Structure :

- **HashSet :** Internally uses HashMap to store the elements.
- **LinkedHashSet :** Internally uses LinkedHashMap to store the elements.
- **TreeSet :** Internally uses TreeMap to store the elements.

Null Elements :

- **HashSet :** Maximum one null element
- **LinkedHashSet :** Maximum one null element.
- **TreeSet :** Doesn't allow even a single null element

Duplicate Elements :

- **HashSet :** Not allowed
- **LinkedHashSet :** Not allowed
- **TreeSet :** Not allowed

Order Of Elements :

- **HashSet :** No order
- **LinkedHashSet :** Insertion order
- **TreeSet :** Elements are placed according to supplied Comparator or in natural order if no Comparator is supplied.

Synchronization :

- **HashSet :** Not synchronized
- **LinkedHashSet :** Not synchronized
- **TreeSet :** Not synchronized

Performance :

- **HashSet :** Insertion -> O(1), Removal -> O(1), Retrieval -> O(1)
- **LinkedHashSet :** Insertion -> O(1), Removal -> O(1), Retrieval -> O(1)
- **TreeSet :** Insertion -> O(log(n)), Removal -> O(log(n)), Retrieval -> O(log(n))

When to use?

- **HashSet :** Use it when you want only unique elements without any order.
- **LinkedHashSet :** Use it when you want only unique elements in insertion order.
- **TreeSet :** Use it when you want only unique elements in some specific order.

Map

Intro :

- Map stores the data in the form of key-value pairs where each key is associated with a value.
- Map interface is part of Java collection framework but it doesn't inherit Collection interface.

Popular Implementations :

- HashMap, LinkedHashMap And TreeMap

Internal Structure :

- **HashMap :** It internally uses an array of buckets where each bucket internally uses linked list to hold the elements.
- **LinkedHashMap :** Same as HashMap but it additionally uses a doubly linked list to maintain insertion order of elements.
- **TreeMap :** It internally uses Red-Black tree.

Null Elements :

- HashMap : Only one null key and can have multiple null values
- LinkedHashMap : Only one null key and can have multiple null values.
- TreeMap : Doesn't allow even a single null key but can have multiple null values.

Duplicate Elements :

- **HashMap :** Doesn't allow duplicate keys but can have duplicate values.
- **LinkedHashMap :** Doesn't allow duplicate keys but can have duplicate values.
- **TreeMap :** Doesn't allow duplicate keys but can have duplicate values.

Order Of Elements :

- **HashMap :** No Order
- **LinkedHashMap :** Insertion Order
- **TreeMap :** Elements are placed according to supplied Comparator or in natural order of keys if no Comparator is supplied.

Synchronization :

- **HashMap :** Not synchronized
- **LinkedHashMap :** Not Synchronized
- **TreeMap :** Not Synchronized

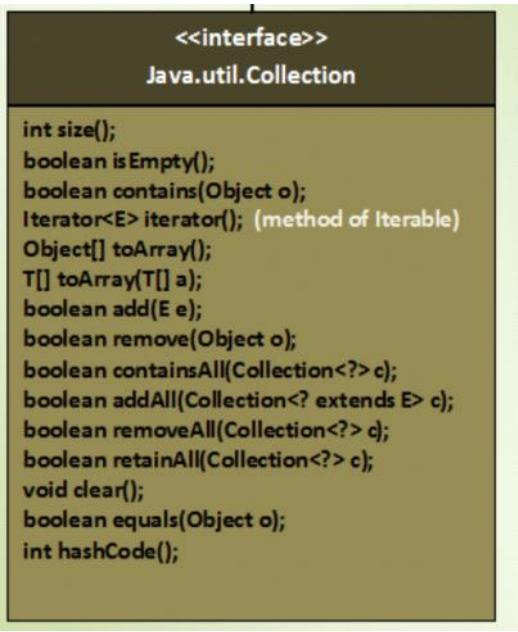
Performance :

- **HashMap :** Insertion -> O(1), Removal -> O(1), Retrieval -> O(1)
- **LinkedHashMap :** Insertion -> O(1), Removal -> O(1), Retrieval -> O(1)
- **TreeMap :** Insertion -> O(log(n)), Removal -> O(log(n)), Retrieval -> O(log(n))

When to use?

- **HashMap :** Use it if you want only key-value pairs without any order.
- **LinkedHashMap :** Use it if you want key-value pairs in insertion order.
- **TreeMap :** Use it when you want key-value pairs sorted in some specific order.

Methods in Collection Interface

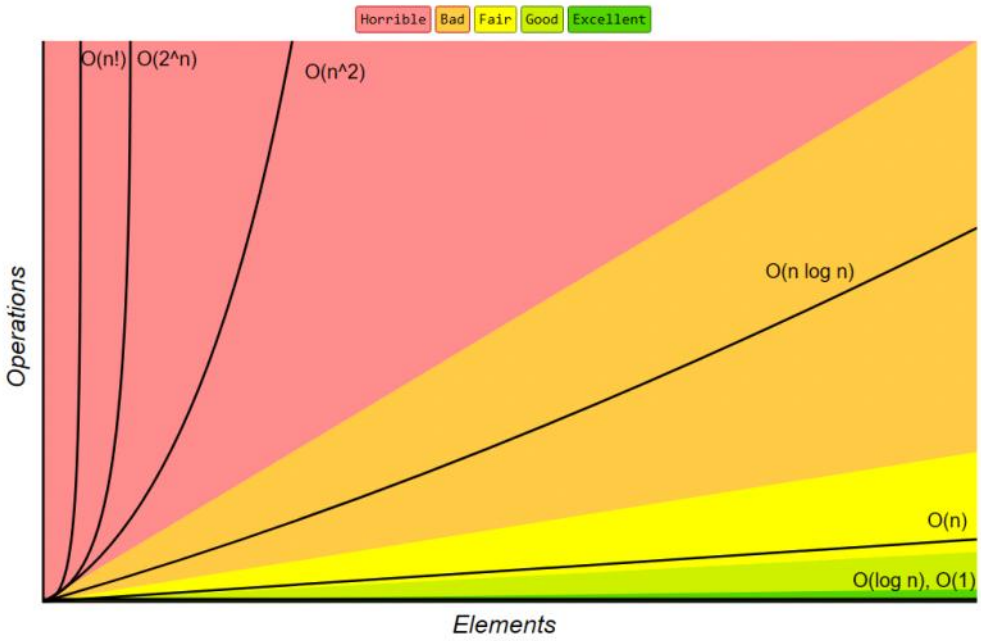

<<interface>>
Java.util.Collection
int size();
boolean isEmpty();
boolean contains(Object o);
Iterator<E> iterator(); (method of Iterable)
Object[] toArray();
T[] toArray(T[] a);
boolean add(E e);
boolean remove(Object o);
boolean containsAll(Collection<?> c);
boolean addAll(Collection<? extends E> c);
boolean removeAll(Collection<?> c);
boolean retainAll(Collection<?> c);
void clear();
boolean equals(Object o);
int hashCode();

Big O

28 February 2024 08:34

Omega (Ω) – Best Case
Theta (Θ) - Average Case
Omicron-Big O (O) - Worst Case

Notation	Name	Description	Example
O(1)	Constant Time	Doesn't depend on the size of the data set.	Accessing an array element by its index.
O(log n)	Logarithmic Time:Divide and Conquer	Splits the data in each step (divide and conquer).	Binary search.
O(n)	Linear Time: Propotional	Directly proportional to the data set size.	Looping through an array.
O(n log n)	Linearithmic Time	Splits and sorts or searches data.	Merge sort, quick sort.
O(n^2)	Polynomial Time: Loop within Loop	Nested loops for each power of n.	Bubble sort (O(n^2)).



Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

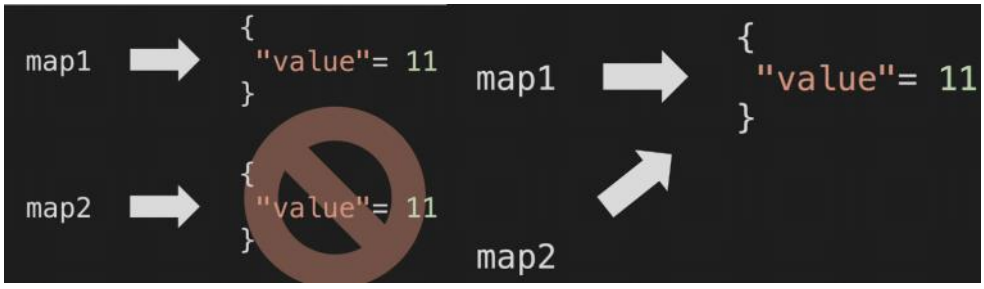
Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	Θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	Θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	Θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	Θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	Θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)

Pointers

27 March 2024 10:59

```
int num1 = 11;
int num2 = num1;
System.out.println(num1); // 11
System.out.println(num2); // 11

num1 = 22;
System.out.println(num1); // 22
System.out.println(num2); // 11


Map<String, Integer> hm1 = new HashMap<>();
Map<String, Integer> hm2 = new HashMap<>();
hm1.put("Value", 1);
hm2 = hm1;

System.out.println(hm1); // {Value=1}
System.out.println(hm2); // {Value=1}

hm1.put("Value", 2); // or hm2.put("Value", 2);

System.out.println(hm1); // {Value=2}
System.out.println(hm2); // {Value=2}
```



- In the non-pointer example, num1 and num2 are independent variables. Changing num1 does not affect num2.
- In the pointer example, hm1 and hm2 reference the same HashMap object. Changes to the map via hm1 affect hm2 and vice versa.

List Interface

02 June 2021 09:34

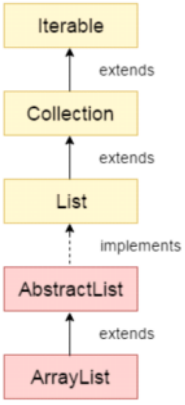
Stores ordered collection of objects with possible duplicates.

```
Additional Methods Of List :
E get(int index);
E set(int index, E element);
void add(int index, E element);
E remove(int index);
Int indexOf(Object o);
int lastIndexOf(Object o);
ListIterator<E> listIterator();
ListIterator<E> listIterator(int index);
List<E> subList(int fromIndex, int toIndex);
```

ArrayList	LinkedList
• Uses dynamic array. • Maintains insertion order, non-synchronized. • Allows random access. • ArrayList is not efficient for manipulation because too much shifting is required. • ArrayList is better to store and fetch data. • ArrayList takes less memory overhead as it stores only object	• Uses doubly linked list. • Maintains insertion order, non-synchronized. • Fast manipulation due to no shifting. • LinkedList does not provide random access. • LinkedList takes more memory overhead, as it stores the object as well as the address of that object

ArrayList

17 June 2018 11:04



- Can contain duplicate elements, Maintains insertion order, Nonsynchronized.
- Allows random access (index basis).
- Manipulation is slow due to required shifting on removal.
- Initial Virtual Capacity: 10, when this is reached new array is created using formula n+(n/2)+1=16.
- ArrayList<String> list2 = new ArrayList<String>(20); //Initial Capacity=20
- ArrayList<String> list = new ArrayList<String>(); list.ensureCapacity(20);
- trimToSize() method is used to trim the capacity of arrayList to the current size of ArrayList
- Cannot store primitive types directly; requires wrapper classes.
- Supports adding, getting, iterating, and removing elements.
- Efficient for adding/removing from end, fast access for searching.
- Positional Access: Operations based on numerical positions: add(int index, Object o), addAll(int index, Collection c), remove(int index), get(int index), set(int index, Object new).
- Search: Methods for finding elements: indexOf(Object o), lastIndexOf(Object o).

Function	**Usage Example**
add(E e)	ArrayList<String> list = new ArrayList<>(); list.add("Apple");
add(int index, E element)	list.add(1, "Banana");
get(int index)	String item = list.get(0);
set(int index, E element)	list.set(0, "Orange");
remove(int index)	list.remove(1);
remove(Object o)	list.remove("Apple"); list.remove(Integer.valueof(30);
clear()	list.clear();
size()	int size = list.size();
isEmpty()	boolean empty = list.isEmpty();
contains(Object o)	boolean hasApple = list.contains("Apple");
indexOf(Object o)	int index = list.indexOf("Apple");
lastIndexOf(Object o)	int lastIndex = list.lastIndexOf("Apple");
toArray()	Object[] array = list.toArray();
subList(int fromIndex, int toIndex)	List<String> subList = list.subList(1, 3);
addAll(Collection<? extends E> c)	list.addAll(anotherList);
addAll(int index, Collection<? extends E> c)	list.addAll(1, anotherList);
retainAll(Collection<?> c)	list.retainAll(anotherList);
removeAll(Collection<?> c)	list.removeAll(anotherList);
iterator()	Iterator<String> it = list.iterator(); while (it.hasNext()) { String item = it.next(); }

<table>
<tr><td>Synchronized ArrayList</td><td>• Better suited for write-heavy scenarios.
• Locks the entire list during read and write operations.
• Can be less efficient for frequent read operations.
• Requires careful synchronization to avoid deadlocks and other concurrency issues.
<table>
<tr><td>Creation</td><td>List<String> syncList=Collections.synchronizedList(new ArrayList<String>());</td></tr>
<tr><td>Adding Elements</td><td>syncList.add("Saurav");</td></tr>
<tr><td>Get 1 Element</td><td>String element = synchronizedList.get(index);</td></tr>
<tr><td>Iterating</td><td>for (String element : synchronizedList) {
System.out.println(element);
}</td></tr>
<tr><td>Updating</td><td>synchronized (synchronizedList) {
synchronizedList.set(index, newValue);
}</td></tr>
<tr><td>Removing data</td><td>synchronized (synchronizedList) {
synchronizedList.remove("Cherry");
}</td></tr>
</table></td></tr>
<tr><td>CopyOnWriteArrayList</td><td>For scenarios where read operations are more frequent than write operations, consider using CopyOnWriteArrayList. It's a thread-safe alternative that creates a copy of the underlying array on each write operation, ensuring thread safety without the overhead of locking the entire list.</td></tr>
</table>

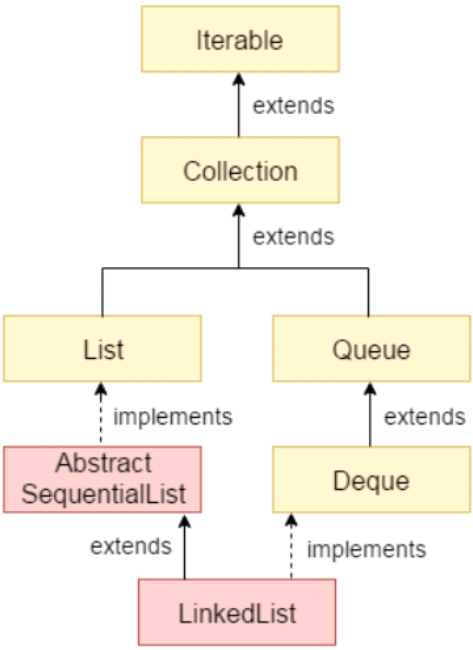
	• Creates a copy of the underlying array on write operations.
	• More efficient for read-heavy workloads.
	• Can be less efficient for frequent write operations due to the overhead of copying.
	
	CopyOnWriteArrayList<String> empList=new CopyOnWriteArrayList<>();
	empList.add("Tom");
	empList.add("Steve");
	empList.forEach(System.out::println);//No Sync Method needed, list is threadsafe
	list.remove("Banana");

ArrayList to Array	String[] array = new String[] {"ANDROID", "JSP", "JAVA", "STRUTS", "HADOOP", "JSF"}; ArrayList<String> list = new ArrayList<String>(Arrays.asList(array)); String[] array = new String[] {"ANDROID", "JSP", "JAVA", "STRUTS", "HADOOP", "JSF"}; ArrayList<String> list = new ArrayList<String>(); Collections.addAll(list, array); List<Object> list = Arrays.stream(array).collect(Collectors.toList());
Array to ArrayList	String[] array = new String[list.size()]; list.toArray(array);

LinkedList

24 July 2018 21:25

- Linked Lists are the data structures where elements are not stored contiguously.
- Each element (node) has a data part and an address part, linked using pointers.
- Nodes are dynamic and allow easy insertions and deletions, preferred over arrays.
- Nodes cannot be accessed directly; traversal starts from the head to reach a desired node.
- Head and tail point to the first and last elements respectively. The last node points to Null.
- Head stores address of first Node.
- Can contain duplicate elements.
- Maintains insertion order.
- Non-synchronized.
- Fast manipulation as no shifting is needed.
- Can be used as a list, stack, or queue.



	Linked List	Array List
Append	O(1)	O(1)
Remove Last	O(n)	O(1)
Prepend	O(1)	O(n)
Remove First	O(1)	O(n)
Insert	O(n)	O(n)
Remove	O(n)	O(n)
Lookup by Index	O(n)	O(1)
Lookup by Value	O(n)	O(n)

Next has reference of next element

```
head= {
        "value"= 11,
        "next"= {
                "value"= 3,
                "next"= {
                        "value"= 23,
                        "next"= {
                                "value"= 7,
                                "next"= {
                                        "value"= 4,
tail=                                   "next"= null
                                        }
                                }
                        }
                }
        }
```

LinkedList	public class LinkedList {
	private Node head, tail;
	private int length;
	
	public LinkedList(int value) {
	Node newNode = new Node(value);//Creates a New Node
	head = newNode;
	tail = newNode;
	length = 1;
	}

	public static class Node {
	int value;
	Node next;
	
	Node(int value) {
	this.value = value;
	}
	}
	}

printList	public void printList() {
	Node temp = head;
	while (temp != null) {
	System.out.println("List Member Value -> " + temp.value);
	temp = temp.next;
	}
	}

Append	public void append(int value) {
	Node newNode = new Node(value);
	if (length == 0) {
	head = newNode;
	} else {
	tail.next = newNode;
	}
	tail = newNode;
	length++;
	System.out.println(value + " added to the List");
	}

removeLastElement	public Node removeLastElement() {
	if (length == 0) return null;
	
	Node temp = head;
	Node pre = head;
	
	while (temp.next != null) {
	pre = temp;
	temp = temp.next;
	}
	tail = pre;
	tail.next = null;
	length--;
	if (length == 0) {
	head = null;
	tail = null;
	}
	return temp;
	}

Prepend	public void prepend(int value) {
	Node newNode = new Node(value);
	if (length == 0) {
	head = newNode;
	tail = newNode;
	} else {
	newNode.next = head;
	head = newNode;
	}
	length++;
	}

removeFirst	public Node removeFirst() {
	if (length == 0) {
	return null;
	}
	
	Node removedNode = head;
	head = head.next;
	removedNode.next = null;
	length--;
	
	if (length == 0) {

	tail = null;
	}
	
	return removedNode;
	}

Get	public Node get(int index) {
	if (index < 0 || index >= length) return null;
	
	Node temp = head;
	for (int i = 0; i < index; i++) {
	temp = temp.next;
	}
	return temp;
	}

Set	public boolean set(int value, int index) {
	Node temp = get(index);
	if (temp != null) {
	temp.value = value;
	return true;
	}
	return false;
	}

Insert	public boolean insert(int value, int index) {
	if (index < 0 || index > length) // Updated index check
	return false;
	
	if (index == 0) {
	prepend(value);
	return true;
	}
	
	if (index == length) { // Append at the end
	append(value);
	return true;
	}
	
	Node newNode = new Node(value);
	Node temp = get(index - 1);
	newNode.next = temp.next;
	temp.next = newNode;
	length++;
	return true;
	}

Remove	public Node remove(int index) {
	if (index < 0 || index >= length) return null;
	
	if (index == 0) {
	return removeFirst();
	}
	
	if (index == length - 1) {
	return removeLastElement();
	}
	
	Node prev = get(index - 1);
	Node temp = prev.next;
	
	prev.next = temp.next;
	temp.next = null;
	length--;
	return temp;
	}

Reverse	public void reverse() {
	Node temp = head;
	head = tail;
	tail = temp;
	Node after;
	Node before = null;

```
    for (int i = 0; i < length; i++) {
        after = temp.next;
        temp.next = before;
        before = temp;
        temp = after;
    }
}
```

Queue

02 June 2021 09:47

<table>
<tr><td>Queue</td><td>A queue is a linear data structure that follows the First In First Out (FIFO) principle, where the element added first will be the first one to be removed. Imagine a queue in a supermarket: the first person to join the queue is the first one to be served. Cannot store null but can store duplicates.
<table>
<tr><td>Key Operations</td><td>Example</td></tr>
<tr><td>Enqueue (Add)</td><td>Purpose: Adds an element to the end of the queue.

Example: queue.add(element)</td></tr>
<tr><td>Dequeue (Remove)</td><td>Purpose: Removes the element from the front of the queue.

Example: queue.remove()</td></tr>
<tr><td>Peek/Front:</td><td>Purpose: Returns the element at the front of the queue without removing it.

Example: queue.peek()</td></tr>
<tr><td>IsEmpty</td><td>Purpose: Checks if the queue is empty.

Example: queue.isEmpty()</td></tr>
<tr><td>Size</td><td>Purpose: Returns the number of elements in the queue.

Example: queue.size()</td></tr>
</table></td></tr>
<tr><td>Deque</td><td>Deque interface extends the Queue interface. In Deque, we can remove and add the elements from both the side. Deque stands for a double-ended queue which enables us to perform the operations at both the ends. Allows both LIFO and FIFO. Can store null and duplicates
<table>
<tr><td>Operation</td><td></td><td>Throws an exception if operation fails.</td><td>Returns null or false if operation fails.</td></tr>
<tr><td>Insertion</td><td>Front End</td><td>addFirst()</td><td>offerFirst()</td></tr>
<tr><td></td><td>Rear End</td><td>addLast()</td><td>offerLast()</td></tr>
<tr><td>Retrieval</td><td>Front End</td><td>getFirst()</td><td>peekFirst()</td></tr>
<tr><td></td><td>Rear End</td><td>getLast()</td><td>peekLast()</td></tr>
<tr><td>Retrieval And Removal</td><td>Front End</td><td>removeFirst()</td><td>pollFirst()</td></tr>
<tr><td></td><td>Rear End</td><td>removeLast()</td><td>pollLast()</td></tr>
</table></td></tr>
<tr><td>PriorityQueue</td><td>Elements are ordered according to their natural ordering or by a Comparator provided at queue construction time.
PriorityQueue does not permit null elements.
it is not synchronized
Based on Complete Binary Tree
Elements with higher priority are dequeued before elements with lower priority. In default PQ, smallest element will be removed 1st

• PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.reverseOrder());
• boolean add(E e): Inserts the specified element into this priority queue.
• boolean offer(E e): Inserts the specified element into this priority queue. Does not throw an exception if the insertion fails due to capacity restrictions, making it a safer option when you're not sure about the capacity.
• E poll(): Retrieves and removes the head of this queue, or returns null if this queue is empty.
• E peek(): Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.
• int size(): Returns the number of elements in this priority queue.
• void clear(): Removes all of the elements from this priority queue.
• Boolean remove(E e): Removes a specific element from the queue.</td></tr>
</table>

Feature	Stack	Queue	Deque (Double-ended Queue)
Data Structure Type	Linear	Linear	Linear
Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)	Allows insertion and deletion from both ends
Basic Operations	push() (add to top), pop() (remove from top)	enqueue() (add to rear), dequeue() (remove from front)	Insertions: enqueueFront(), enqueueRear(); Removals: dequeueFront(), dequeueRear()
Insertion	At the top	At the rear	At both front and rear
Deletion	From the top	From the front	From both front and rear
Use Cases	Undo mechanisms, browser history, recursive function calls	Order processing, scheduling, printers, BFS	Caching, undo-redo, palindromic checks
Direct Access	Not allowed	Not allowed	Not allowed
Storage	Typically array-based or linked list	Typically array-based or linked list	Typically array-based or linked list

Stack

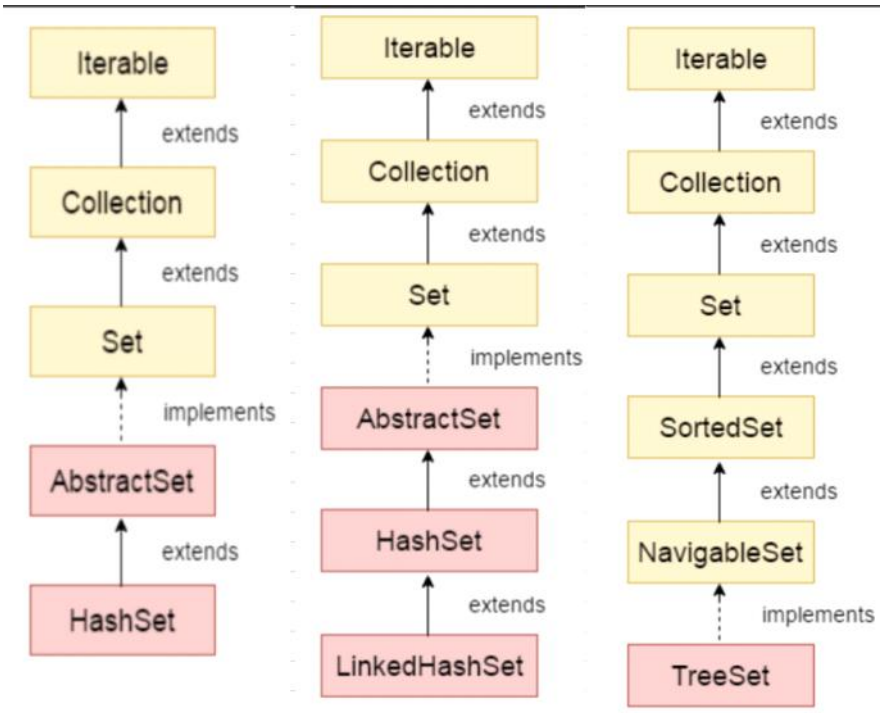
28 November 2024 09:00

A stack is a linear data structure that follows the Last In First Out (LIFO) principle. This means the last element added to the stack will be the first one to be removed. Imagine a stack of plates: you add a plate on top, and it's also the first one you take off.

Key Operations	Purpose
Push	Purpose: Adds an element to the top of the stack. Example: stack.push(element)
Pop	Purpose: Removes the element from the top of the stack. Example: stack.pop()
Peek	Purpose: Returns the element at the top of the stack without removing it. Example: stack.peek()
isEmpty	Purpose: Checks if the stack is empty. Example: stack.isEmpty()
size	Purpose: Returns the number of elements in the stack. Example: stack.size()

Set Interface

02 June 2021 10:39

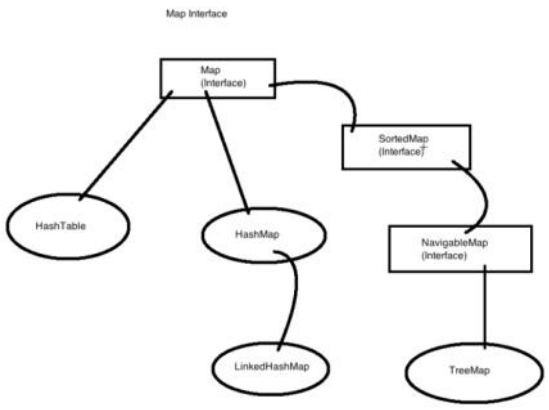


Set	Present in java.util package. It extends the Collection interface. It represents the unordered set of elements which doesn't allow us to store the duplicate items. We can store at most one null value in Set. Stores Unique Element and it will grow dynamically when elements are added to it.
HashSet	Implements Set Interface. It represents the collection that uses a hash table for storage. Hashing is used to store the elements in the HashSet and contains unique items. • HashSet stores the elements by using a mechanism called hashing. • HashSet contains unique elements only. • HashSet allows 1 null value. • HashSet class is non synchronized. • HashSet doesn't maintain the insertion order. Elements are inserted on the basis of their hashcode. • HashSet is the best approach for search operations. • The initial default capacity of HashSet is 16, and the load factor is 0.75. Size doubles after that and rehashing occurs. Methods in HashSet: • boolean add(E e): Used to add the specified element if it is not present, if it is present then return false. • void clear(): Used to remove all the elements from set. • boolean contains(Object o): Used to return true if an element is present in set. • boolean remove(Object o): Used to remove the element if it is present in set. • Iterator iterator(): Used to return an iterator over the element in the set. • boolean isEmpty(): Used to check whether the set is empty or not. Returns true for empty and false for non-empty condition for set. • int size(): Used to return the size of the set. • Object clone(): Used to create a shallow copy of the set. HashSet uses HashMap internally to store it's objects. Whenever you create a HashSet object, one HashMap object associated with it is also created. This HashMap object is used to store the elements you enter in the HashSet. The elements you add into HashSet are stored as keys of this HashMap object. The value associated with those keys will be a constant and the values will be a constant called PRESENT. Whenever we create a HashSet, it internally creates a HashMap and if we insert an element into this HashSet using add() method, it actually call put() method on internally created HashMap object with element you have specified as it's key and constant Object called "PRESENT" as it's value. So we can say that a Set achieves uniqueness internally through HashMap. As we know in a HashMap each key is unique and when we call put(Key, Value) method, it returns the previous value associated with key, or null if there was no mapping for key. So in add() method we check the return value of map.put(key, value) method with null value.

LinkedHashSet	Represents the LinkedList implementation of Set Interface. It extends the HashSet class and implements Set interface. Like HashSet, It also contains unique elements. It maintains the insertion order and permits null elements. The insertion order of elements into LinkedHashMap are maintained by adding two new fields to this class. They are before and after. These two fields hold the references to previous and next elements. These two fields make LinkedHashMap to function as a doubly linked list. Java LinkedHashSet class is a Hashtable and doubly Linked list implementation of the set interface. It inherits HashSet class and implements Set interface. • Java LinkedHashSet class contains unique elements only like HashSet. • Allows null element. • Java LinkedHashSet class provides all optional set operation and permits null elements. • Java LinkedHashSet class is non synchronized. • Java LinkedHashSet class maintains insertion order.
SortedSet	Alternate of Set interface that provides a total ordering on its elements. The elements of the SortedSet are arranged in the increasing (ascending) order. The SortedSet provides the additional methods that inhibit the natural ordering of the elements.
TreeSet	Implements the Set interface and sorted set interface that uses a tree for storage. Like HashSet, TreeSet also contains unique elements. However, the access and retrieval time of TreeSet is quite fast logn. The elements in TreeSet stored in ascending order. • Java TreeSet class contains unique elements only like HashSet. • Java TreeSet class access and retrieval times are quiet fast. • Java TreeSet class doesn't allow null element. • Java TreeSet class is non synchronized. • Java TreeSet class maintains ascending order.

Map Interface

12 February 2019 22:38

Introduction	 A `Map` in Java stores key-value pairs, with each pair known as an entry. Keys in a Map must be unique. Maps are ideal for searching, updating, or deleting elements based on a key. Key Characteristics of Map Interface: - Uniqueness: Each key is unique and maps to at most one value. - Null Handling: HashMap and LinkedHashMap allow null keys and values; TreeMap does not. - Order: The order of entries depends on the implementation (TreeMap and LinkedHashMap have predictable orders, HashMap does not). - Interfaces and Classes: Implemented via Map and SortedMap interfaces, and HashMap, TreeMap, and LinkedHashMap classes. - Traversal: Maps cannot be directly traversed; use keySet() or entrySet() to convert them to a Set.

Printing Values	for (Map.Entry<String, Integer> entry : map.entrySet()) {
	  System.out.println("Key: " + entry.getKey() + ", Value: " + entry.getValue());
	}
	
	map.forEach((key, value) -> System.out.println("Key: " + key + ", Value: " + value));

Common Methods	1. put(K key, V value)
	  - Inserts the specified value with the specified key in the map.
	  Example: map.put("key1", "value1");
	
	2. get(Object key)
	  - Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
	  Example: String value = map.get("key1");
	
	3. remove(Object key)
	  - Removes the mapping for a key from this map if it is present.
	  Example: map.remove("key1");
	
	4. containsKey(Object key)
	  - Returns true if this map contains a mapping for the specified key.
	  Example: boolean exists = map.containsKey("key1");
	
	5. containsValue(Object value)
	  - Returns true if this map maps one or more keys to the specified value.
	  Example: boolean exists = map.containsValue("value1");
	
	6. keySet()
	  - Returns a Set view of the keys contained in this map.
	  Example: Set<String> keys = map.keySet();
	
	7. values()
	  - Returns a Collection view of the values contained in this map.
	  Example: Collection<String> values = map.values();
	
	8. entrySet()
	  - Returns a Set view of the mappings contained in this map.
	  Example: Set<Map.Entry<String, String>> entries = map.entrySet();
	
	9. isEmpty()
	  - Returns true if this map contains no key-value mappings.
	  Example: boolean isEmpty = map.isEmpty();

	10. size() - Returns the number of key-value mappings in this map. Example: int size = map.size(); 11. clear() - Removes all of the mappings from this map. Example: map.clear(); 12. putAll(Map<? extends K, ? extends V> m) - Copies all of the mappings from the specified map to this map. Example: map.putAll(anotherMap);

HashMap	- Key-Value Storage: Contains values based on the key.
	- Unique Keys: Only unique keys are allowed.
	- Null Handling: May have one null key and multiple null values.
	- Synchronization: Non-synchronized.
	- Order: Does not maintain any order.
	- Capacity and Load Factor:
	- Default initial capacity: 16.
	- Default load factor: 0.75.
	- Threshold: Increases capacity from 16 to 32 after the 12th entry.
	- The capacity of the HashMap is doubled each time it reaches the threshold. i.e the capacity is increased to 25=32, 26=64, 27= 128….. when the threshold is reached.
	
	Hashing: Uses hashing for converting an object into an integer form using the hashCode() method, enabling faster indexing and searches. HashSet uses HashMap internally.
	
	Conversion Of HashMap Keys Into ArrayList
	Set<String> keySet = map.keySet();
	//Creating an ArrayList of keys by passing the keySet
	ArrayList<String> listOfKeys = new ArrayList<String>(keySet);
	
	Conversion Of HashMap Values Into ArrayList :
	Collection<String> values = map.values();
	//Creating an ArrayList of values
	ArrayList<String> listOfValues = new ArrayList<String>(values);
	
	Conversion Of HashMap's Key-Value Pairs Into ArrayList
	Set<Entry<String, String>> entrySet = map.entrySet();
	//Creating an ArrayList of Entry objects
	ArrayList<Entry<String, String>> listOfEntry = new ArrayList<Entry<String, String>>(entrySet);

Hashtable	Java Hashtable class implements a hashtable, which maps keys to values. It inherits Dictionary class and implements the Map interface. • A Hashtable is an array of a list. Each list is known as a bucket. The position of the bucket is identified by calling the ha shcode() method. A Hashtable contains values based on the key. • Java Hashtable class contains unique elements. • Java Hashtable class doesn't allow null key or value. • Java Hashtable class is synchronized. • The initial default capacity of Hashtable class is 11 whereas loadFactor is 0.75.

Working of HashMap

03 June 2021 11:39

HashMap is a part of the Java collection framework. It uses a technique called Hashing. It implements the map interface. It s tores the data in the pair of Key and Value. HashMap contains an array of the nodes, and the node is represented as a class. It uses an array and LinkedList data s tructure internally for storing Key and Value. There are four fields in HashMap.

The whole HashMap data structure is based on the principle of Hashing. Hashing is nothing but the function or algorithm or me thod which when applied on any object/variable returns an unique integer value representing that object/variable. This unique integer value is called hash c ode. Hash function or simply hash said to be the best if it returns the same hash code each time it is called on the same object. Two objects can have same hash cod e.

Before understanding the internal working of HashMap, you must be aware of hashCode() and equals() method.

- **equals():** It checks the equality of two objects. It compares the Key, whether they are equal or not. It is a method of the Object class. It can be overridden. If you override the equals() method, then it is mandatory to override the hashCode() method.
- **hashCode():** This is the method of the object class. It returns the memory reference of the object in integer form. The value received from the method is used as the bucket number. The bucket number is the address of the element inside the map. Hash code of null Key is 0.
- **Buckets:** Array of the node is called buckets. Each node has a data structure like a LinkedList. More than one node can share the same bucket. It may be different in capacity.

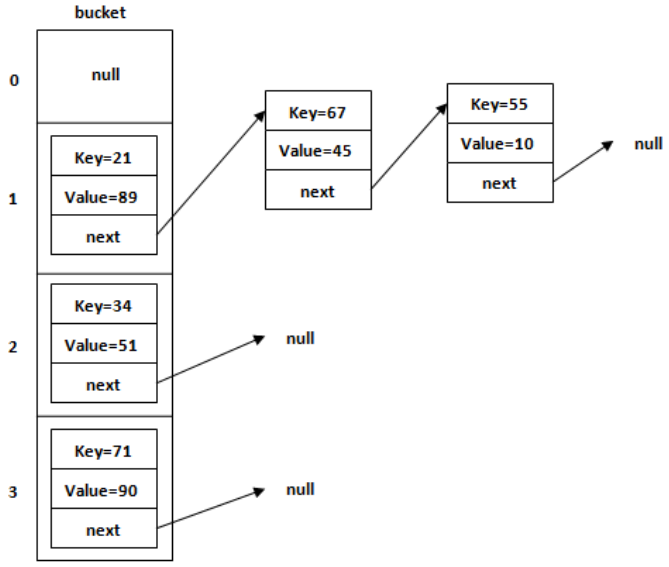


Figure: Allocation of nodes in Bucket

Insert Key, Value pair in HashMap

We use put() method to insert the Key and Value pair in the HashMap. The default size of HashMap is 16 (0 to 15).

```
HashMap<String, Integer> map = new HashMap<>();
map.put("Aman", 19);
map.put("Sunny", 29);
map.put("Ritesh", 39);
```

When we call the put() method, then it calculates the hash code of the Key "Aman." Suppose the hash code of "Aman" is 2657860 . To store the Key in memory, we have to calculate the index.

Calculating Index

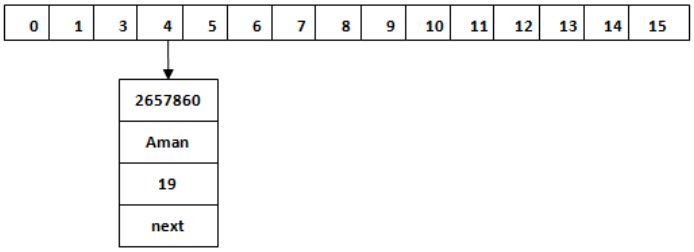
Index minimizes the size of the array. The Formula for calculating the index is:

Index = hashcode(Key) % (n-1) // n is size of hashmap

Where n is the size of the array. Hence the index value for "Aman" is:

Index = 2657860 % (16-1) = 4

The value 4 is the computed index value where the Key and value will store in HashMap.

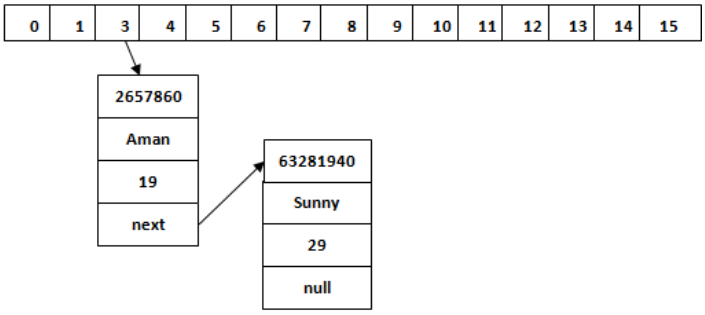


Hash Collision

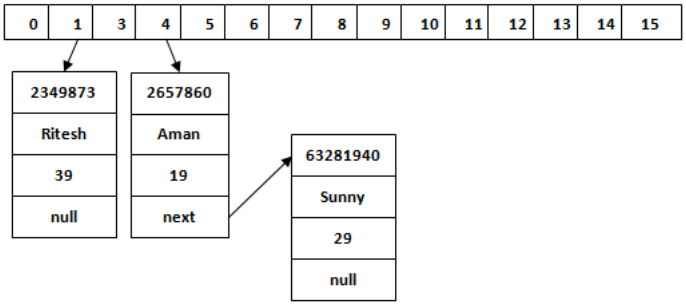
This is the case when the calculated index value is the same for two or more Keys. Let's calculate the hash code for another Key "Sunny." Suppose the hash code for "Sunny" is 63281940. To store the Key in the memory, we have to calculate index by using the index formula.

Index=63281940 & (16-1) = 4

The value 4 is the computed index value where the Key will be stored in HashMap. In this case, equals() method check that bot h Keys are equal or not. If Keys are same, replace the value with the current value. Otherwise, connect this node object to the existing node object through the L inkedList. Hence both Keys will be stored at index 4.



Similarly, we will store the Key "Ritesh." Suppose hash code for the Key is 2349873. The index value will be 1. Hence this Ke y will be stored at index 1.



get() method in HashMap

get() method is used to get the value by its Key. It will not fetch the value if you don't know the Key. When get(K Key) meth od is called, it calculates the hash code of the Key.

Suppose we have to fetch the Key "Aman." The following method will be called.

map.get(new Key("Aman"));

It generates the hash code as 2657860. Now calculate the index value of 2657860 by using index formula. The index value will be 4, as we have calculated above. get() method search for the index value 4. It compares the first element Key with the given Key. If both keys are equal, then it returns the value else check for the next element in the node if it exists. In our scenario, it is found as the first element of the node and return the value 19.

Let's fetch another Key "Sunny."
The hash code of the Key "Sunny" is 63281940. The calculated index value of 63281940 is 4, as we have calculated for put() me thod. Go to index 4 of the array and compare the first element's Key with the given Key. It also compares Keys. In our scenario, the given Key is the second e lement, and the next of the node is null. It compares the second element Key with the specified Key and returns the value 29. It returns null if the next of the node is null.

Interview

02 March 2019 08:36

Collection is a root level interface of the Java Collection Framework. It is a set of classes and interfaces designed for storing and manipulating object data. It includes classes like ArrayList, Vector, Stack, HashSet, and interfaces like List, Queue, and Set.

Collections is an utility class in java.util package. It consists of only static methods which are used to operate on objects of type Collection.

- Collections.max():This method returns maximum element in the specified collection.
- Collections.min():This method returns minimum element in the given collection.
- Collections.sort():This method sorts the specified collection.
- Collections.copy():This method copies all elements from one collection to another collection.
- Collections.reverse():This method reverses the order of elements in the specified collection.

Array to List: new ArrayList<>(Arrays.asList(array));
List to Array: String[] array = fruitList.toArray(new String[fruitList.size()]);

Array vs. Collection:
- Arrays have a fixed size, while Collections can dynamically change their size.
- Collections provide ready-made methods for sorting and searching, unlike arrays.

Iterator vs. ListIterator:
1	Forward direction only	Forward and backward direction
2	Used in List, Set, Queue	Used in List only
3	Can perform remove operation	Can perform add, remove, and set operations

List vs. Set:
- List can contain duplicate elements; Set includes unique items.
- List is an ordered collection with insertion order maintained; Set is unordered.
- List allows n number of null values; Set allows only a single null value.

HashSet vs. TreeSet:
- HashSet maintains no order; TreeSet maintains ascending order.
- HashSet implemented by hash table; TreeSet implemented by a Tree structure.
- HashSet performs faster than TreeSet.
- HashSet backed by HashMap; TreeSet backed by TreeMap.

HashMap vs. TreeMap:
- HashMap maintains no order; TreeMap maintains ascending order.
- HashMap implemented by hash table; TreeMap implemented by a Tree structure.
- HashMap can be sorted by Key or value; TreeMap can be sorted by Key.
- HashMap may contain a null key with multiple null values; TreeMap cannot hold a null key but can have multiple null values.

HashMap	Hashtable
HashMap is **non synchronized**.	Hashtable is **synchronized**.
HashMap **allows one null key and multiple null values**.	Hashtable **doesn't allow any null key or value**.
HashMap is a **new class introduced in JDK 1.2**.	Hashtable is a **legacy class**.
HashMap is **fast**.	Hashtable is **slow**.
We can make the HashMap as synchronized by calling this code Map m = Collections.synchronizedMap(hashMap);	Hashtable is internally synchronized and can't be unsynchronized.
HashMap is **traversed by Iterator**.	Hashtable is **traversed by Enumerator and Iterator**.
Iterator in HashMap is **fail-fast**.	Enumerator in Hashtable is **not fail-fast**.
HashMap inherits **AbstractMap** class.	Hashtable inherits **Dictionary** class.

Synchronizing List, Set, and Map elements:
Collections class provides methods to make List, Set, or Map elements synchronized:
- public static List synchronizedList(List l){}
- public static Set synchronizedSet(Set s){}
- public static SortedSet synchronizedSortedSet(SortedSet s){}
- public static Map synchronizedMap(Map m){}
- public static SortedMap synchronizedSortedMap(SortedMap m){}

Advantages of generic collection:
1. No need for typecasting.
2. Type-safe and checked at compile time.
3. Confirms code stability by making it bug-detectable at compile time.

Fail-fast:
The Iterator in Java that immediately throws ConcurrentModificationException if any structural modification occurs is called a Fail-fast iterator. It does not require extra space in memory.

Array vs. ArrayList:
- Arrays have a fixed size; ArrayList is not of fixed size and can change dynamically.
- Arrays are of static type; ArrayList is of dynamic size.
- Arrays can store primitive data types and objects; ArrayList can only store objects.

Length of Array vs. size of ArrayList:
- Array length is obtained using the property of length; ArrayList uses size() method.

Reverse ArrayList:
- Use Collections.reverse() method.

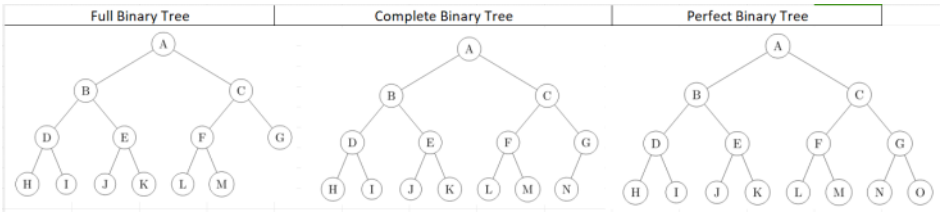
Sort ArrayList in descending order:
- Use Collections.sort() method with Collections.reverseOrder() comparator

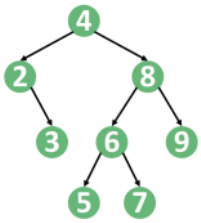
BST

24 November 2024 09:18

<table>
<tr><td>Tree</td><td>It is a data structure that is used to represent the data in hierarchical form. It can be defined as a collection of objects or entities called as nodes that are linked together to simulate a hierarchy. Tree is a non-linear data structure as the data in a tree is not stored linearly or sequentially. Node without childred is called leaf.

Full Tree: Every node points to either zero or two nodes.
Complete Tree: Nodes are filled from left to right with no gaps.
Perfect Tree: All levels are fully filled with nodes.

 
</td></tr>
<tr><td>BST</td><td>It follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

 
</td></tr>
<tr><td>Space Time Complexity</td><td><table>
<tr><th>Operations</th><th>Best case time complexity</th><th>Average case time complexity</th><th>Worst case time complexity</th><th>Space complexity</th></tr>
<tr><td>Insertion</td><td>O(log n)</td><td>O(log n)</td><td>O(n)</td><td>O(n)</td></tr>
<tr><td>Deletion</td><td>O(log n)</td><td>O(log n)</td><td>O(n)</td><td>O(n)</td></tr>
<tr><td>Search</td><td>O(log n)</td><td>O(log n)</td><td>O(n)</td><td>O(n)</td></tr>
</table></td></tr>
<tr><td></td><td></td></tr>
</table>